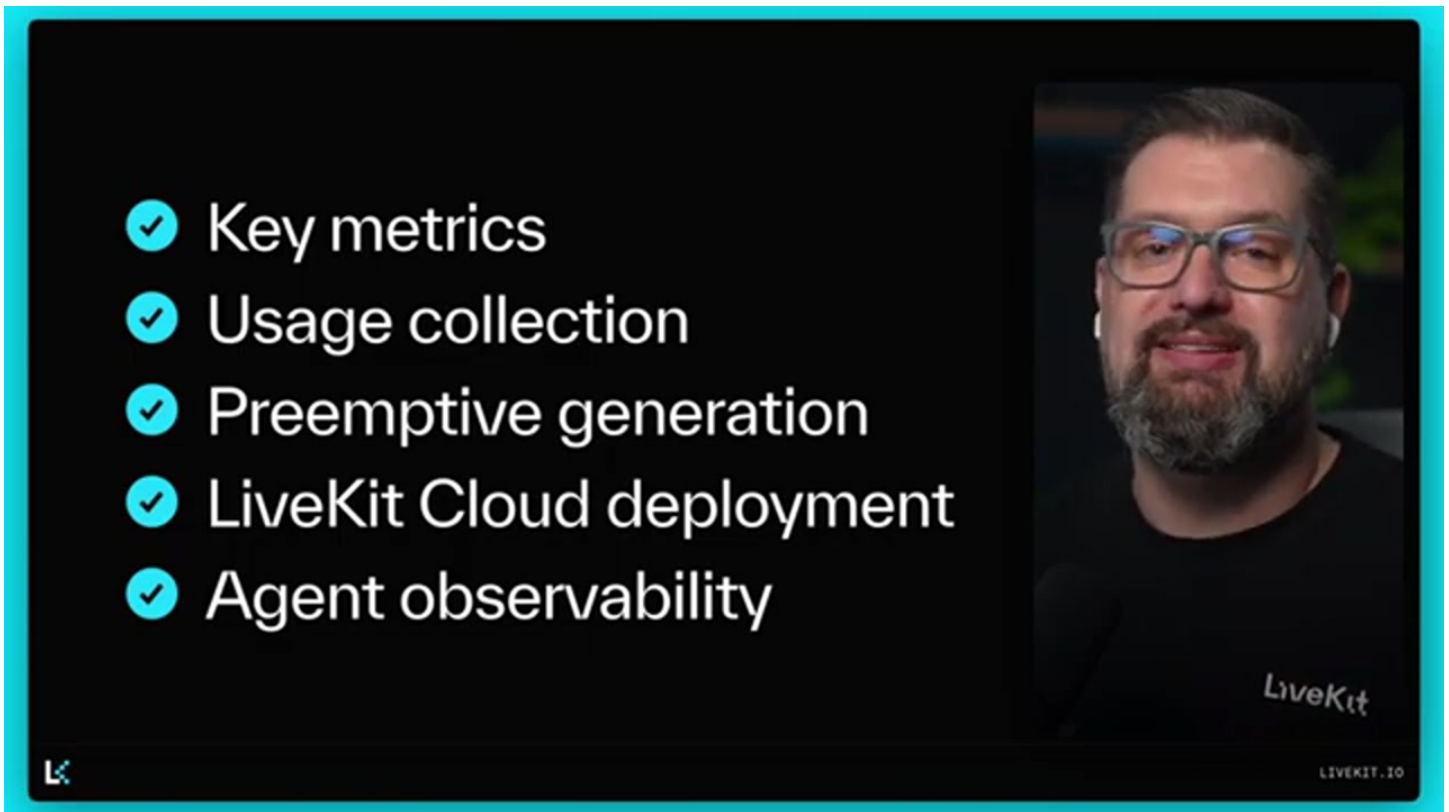


Lesson 5: Tools and MCP Integration

Section: LiveKit Voice AI Course

Summary: Coverage of Lesson 5: Tools and MCP Integration (44:59 - 56:36)

■ [44:59 - 45:19]



- ✓ Key metrics
- ✓ Usage collection
- ✓ Preemptive generation
- ✓ LiveKit Cloud deployment
- ✓ Agent observability

tools and workflows in the next lesson. So, your agent can speak naturally, and you can measure its performance. Now, it's time to give it capabilities beyond conversation tools. Let your agent call external APIs, look up data, and take actions on behalf of users. And without tools, your agent can only respond based on what's in its training data and

Why tools matter



LIVEKIT.IO

on what's in its training data and conversation context, and that's pretty limiting. Tools let you fetch real-time information like weather, stock prices, or order status. They let you take actions in external systems like creating tickets, sending emails, or updating records. You can access your private data from customer databases or

Why tools matter

- Fetch real-time info
- Take actions in external systems
- Access private data



LIVEKIT.IO

private data from customer databases or internal docs. And you can trigger UI updates on the front end via RPC tools. Turn your agent from a chatbot into something that can actually get things done. So Lifegate uses the function tool decorator to expose Python functions to the LLM. The LLM decides when to call

@function_tool decorator



LiveKit

LIVEKIT.IO

the LLM. The LLM decides when to call them based on the conversation context. So let's add a weather lookup tool. This example uses the open Matteo API which is free and doesn't require an API key. Okay, first thing at the top, let's import uh HTTP ex so that we can make HTTP requests

■ [46:19 - 46:39]

```
agent.py x
livekit-voice-agent > agent.py
1  import logging
2
3  from dotenv import load_dotenv
4  from livekit import agents
5  from livekit.agents import Agent, AgentServer, AgentSession, JobContext, room_io
6  from livekit.plugins import noise_cancellation, silero
7  from livekit.plugins.turn_detector.multilingual import MultilingualModel
8  from livekit.agents import llm, stt, tts, inference
9  from livekit.agents import AgentStateChangedEvent, MetricsCollectedEvent, metric
10 import time
11 import httpx
12
13 logger = logging.getLogger(__name__)
14
15 load_dotenv()
16
17
18 class Assistant(Agent):
19     def __init__(self) -> None:
20         super().__init__(
21             instructions="You are an upbeat, slightly sarcastic voice AI for tech support. "
22             "Help the caller fix issues without rambling, and keep replies under 3 sentences.",
23         )
```



ex so that we can make HTTP requests and let's not forget to install that package as well. So, u add httpex. Then we also need to import some more things from lifekit agents. We need the function tool, the run context and the tool error. Let's go down to our instructions and let's update these as well. So, you are an upbeat, slightly

■ [46:39 - 46:59]

```
kit.agents import AgentStateChangedEvent, MetricsCollectedEvent, metrics
kit.agents import function_tool, RunContext, ToolError
me
tpx

logging.getLogger(__name__)

nv()

class InstantAgent(Agent):
    def __init__(self) -> None:
        super().__init__(
            instructions=[
                "You are an upbeat, slightly sarcastic voice AI for tech support. "
                "Help the caller fix issues without rambling, and keep replies under 3 sentences. "
                "You can also look up the weather if asked."
            ],

AgentServer()

def tc_session()
```



well. So, you are an upbeat, slightly sarcastic tech support, all that same stuff. Help the caller fix issues without rambling. Uh, but then we've added you can also look up the weather if asked. So, we're going to add this uh this function right here. And so, let's go ahead and walk through this. So, the the function tool decorator registers this

■ [46:59 - 47:19]



```
agent.py x
livekit-voice-agent > agent.py

19 class Assistant(Agent):
20     def __init__(self) -> None:
21         super().__init__(
22             instructions=(
23                 "You can also look up the weather if asked."
24             ),
25         )
26
27
28
29 @function_tool()
30 async def lookup_weather(
31     self,
32     context: RunContext,
33     location: str,
34 ) -> dict:
35     """Look up current weather for a location.
36
37     Args:
38         location: City name or location to get weather for.
39     """
40     async with httpx.AsyncClient() as client:
41         # First, geocode the location to get coordinates
42         geo_response = await client.get(
43             f"https://ipgeolocation.prl-api.com/v1/search?lat={lat}&lon={lon}"
```

function tool decorator registers this method as a callable tool for the LLM. Then we have the context parameter which is a run context and this provides access to the current session, the speech handle for the interruption detection and any custom user data that you've stored. We've got our location.

■ [47:19 - 47:39]

```
agent.py x
livekit-voice-agent > agent.py

19 class Assistant(Agent):
30     async def lookup_weather(
34         ) -> dict:
35         """Look up current weather for a location.
36
37         Args:
38             location: City name or location to get weather for.
39         """
40         async with httpx.AsyncClient() as client:
41             # First, geocode the location to get coordinates
42             geo_response = await client.get(
43                 "https://geocoding-api.open-meteo.com/v1/search",
44                 params={"name": location, "count": 1},
45             )
46             geo_data = geo_response.json()
47
48             if not geo_data.get("results"):
49                 raise ToolError(f"Could not find location: {location}")
50
51             lat = geo_data["results"][0]["latitude"]
52             lon = geo_data["results"][0]["longitude"]
53             place_name = geo_data["results"][0]["name"]
54
```



you've stored. We've got our location. Now the dock string is critical here. The LLM reads the text to understand what the tool does and when it should be called. When a user says, "What is the weather in Paris?" The LLM scans through all of the available tools, reads each dock string, and decides which one matches the user's intent. A vague dock

■ [47:39 - 47:59]

```
agent.py x
livekit-voice-agent > agent.py
19 class Assistant(Agent):
30     async def lookup_weather(
32         context: RunContext,
33         location: str,
34     ) -> dict:
35         """Look up current weather for a location"""
36
37         Args:
38             location: City name or location to get weather for.
39         """
40         async with httpx.AsyncClient() as client:
41             # First, geocode the location to get coordinates
42             geo_response = await client.get(
43                 "https://geocoding-api.open-meteo.com/v1/search",
44                 params={"name": location, "count": 1},
45             )
46             geo_data = geo_response.json()
47
48             if not geo_data.get("results"):
49                 raise ToolError(f"Could not find location: {location}")
50
51             lat = geo_data["results"][0]["latitude"]
52             lon = geo_data["results"][0]["longitude"]
```



matches the user's intent. A vague dock string means that the LLM might not call your tool when it should, or it might call it incorrectly. And the args section in the dock string is just as important. It tells the LLM what each parameter means and what values are appropriate. For location, here we are specify city name or location to get

■ [47:59 - 48:19]

```
agent.py x
livekit-voice-agent > agent.py

19 class Assistant(Agent):
30     async def lookup_weather(
32         context: RunContext,
33         location: str,
34     ) -> dict:
35         """Look up current weather for a location.
36
37         Args:
38             location: City name or location to get weather for.
39         """
40         async with httpx.AsyncClient() as client:
41             # First, geocode the location to get coordinates
42             geo_response = await client.get(
43                 "https://geocoding-api.open-meteo.com/v1/search",
44                 params={"name": location, "count": 1},
45             )
46             geo_data = geo_response.json()
47
48             if not geo_data.get("results"):
49                 raise ToolError(f"Could not find location: {location}")
50
51             lat = geo_data["results"][0]["latitude"]
52             lon = geo_data["results"][0]["longitude"]
```



specify city name or location to get weather for. So the LLM knows to extract the city from the user's request. Without this description, the LLM might pass the wrong data or misunderstand what the parameter expects. So then here is our HTTP request where we are calling this uh geocod geocoding API from open

■ [48:19 - 48:39]

```
exit voice-agent > agent.py
19 class Assistant(Agent):
30     async def lookup_weather(
34         ) -> dict:
35         """Look up current weather for a location.
36
37         Args:
38             location: City name or location to get weather for.
39         """
40         async with httpx.AsyncClient() as client:
41             # First, geocode the location to get coordinates
42             geo_response = await client.get(
43                 "https://geocoding-api.open-meteo.com/v1/search",
44                 params={"name": location, "count": 1},
45             )
46             geo_data = geo_response.json()
47
48             if not geo_data.get("results"):
49                 raise ToolError(f"Could not find location: {location}")
50
51             lat = geo_data["results"][0]["latitude"]
52             lon = geo_data["results"][0]["longitude"]
53             place_name = geo_data["results"][0]["name"]
54
55             # Get current weather for those coordinates
56             weather_response = await client.get(
```



this uh geocod geocoding API from open mateo and we're going to pass in that location there and then we will basically just return those results. So if we go back down to the bottom here, we're returning the location, the temperature, and the condition. So then the LLM will have that information to then answer our question. Now notice we do have uh the

■ [48:39 - 48:59]

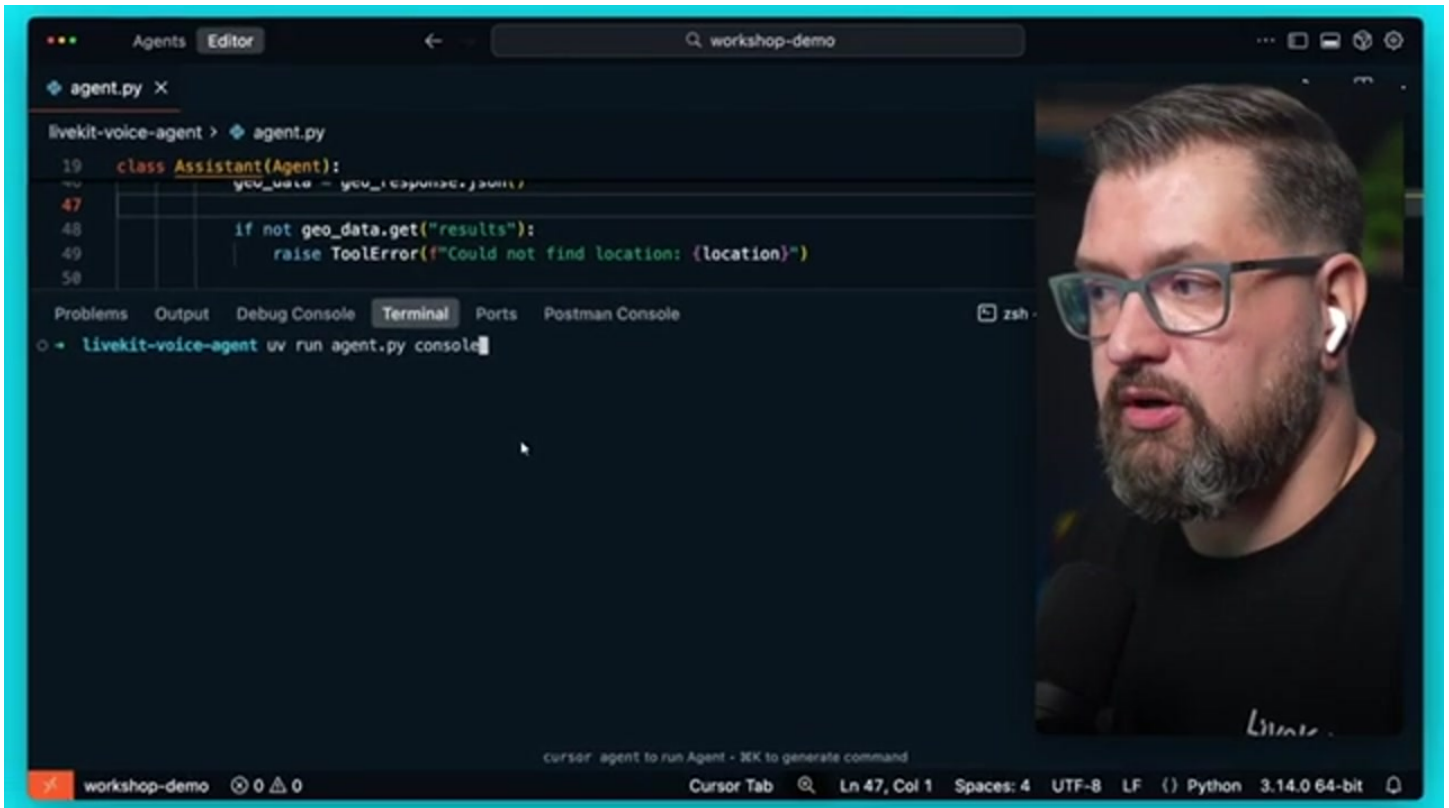
```
agent.py x
livekit-voice-agent > agent.py

19 class Assistant(Agent):
34     ) -> dict:
40     async with httpx.AsyncClient() as client:
42         geo_response = await client.get(
44             params={"name": location, "count": 1},
45         )
46         geo_data = geo_response.json()
47
48         if not geo_data.get("results"):
49             raise ToolError(f"Could not find location: {location}")
50
51         lat = geo_data["results"][0]["latitude"]
52         lon = geo_data["results"][0]["longitude"]
53         place_name = geo_data["results"][0]["name"]
54
55         # Get current weather for those coordinates
56         weather_response = await client.get(
57             "https://api.open-meteo.com/v1/forecast",
58             params={
59                 "latitude": lat,
60                 "longitude": lon,
61                 "current": "temperature_2m,weather_code",
62                 "temperature_unit": "fahrenheit",
```



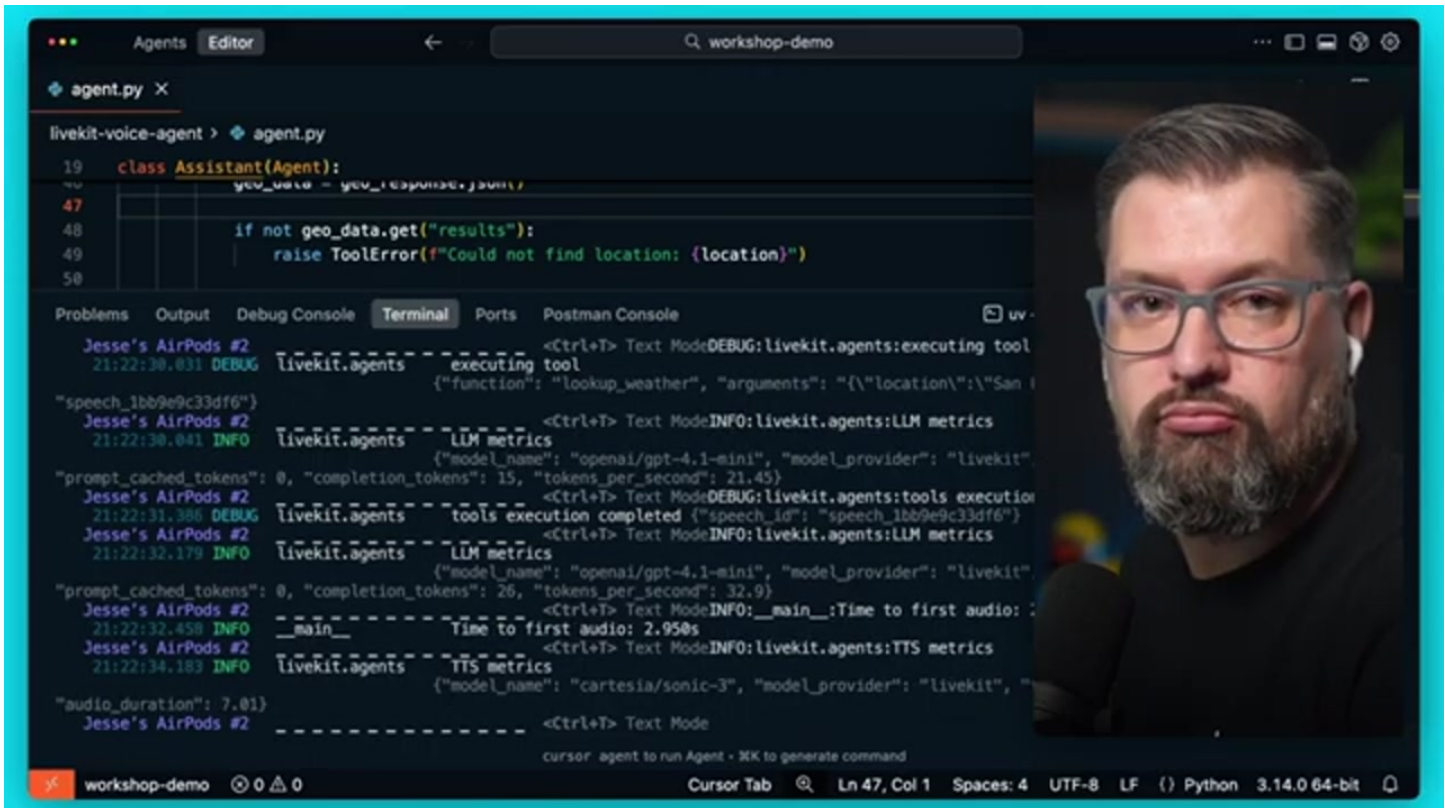
question. Now notice we do have uh the tool error here. So if no location is found, it will return that. And so let's go ahead and test this. Let's see if it works. So make sure that you save this. Let's open up our terminal. And we'll do UV run agent. And then in the console. Now we could uh sync this with our our

■ [48:59 - 49:19]



Now we could uh sync this with our our agent in the cloud which we deployed in the last lesson. Uh but for simple testing and development we can stay here in the console. Hey uh so what's what's the weather like in San Francisco?
>> Currently it's 49.1 degrees Fahrenheit in San Francisco with clear conditions.

■ [49:19 - 49:39]



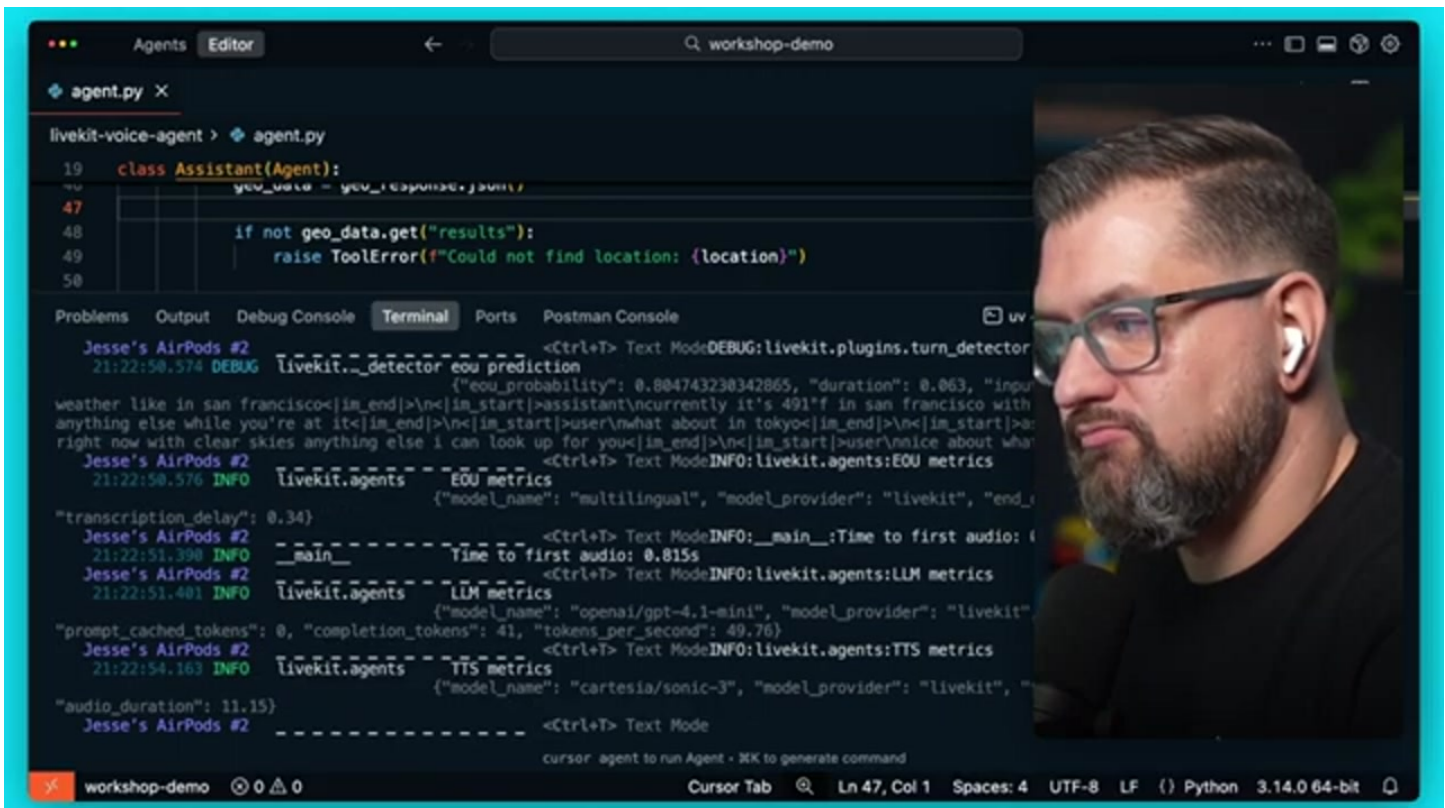
```
agent.py X
livekit-voice-agent > agent.py
19 class Assistant(Agent):
20     you_said = you_response.json()
47
48     if not geo_data.get("results"):
49         raise ToolError(f"Could not find location: {location}")
50

Problems Output Debug Console Terminal Ports Postman Console
Jesse's AirPods #2 21:22:30.831 DEBUG livekit.agents executing tool
{"function": "lookup_weather", "arguments": "{\\"location\\":\\"San Francisco\\"}"}
"speech_id": "speech_1bb9e9c33df6"}
Jesse's AirPods #2 21:22:30.841 INFO livekit.agents LLM metrics
{"model_name": "openai/gpt-4.1-mini", "model_provider": "livekit", "prompt_cached_tokens": 0, "completion_tokens": 15, "tokens_per_second": 21.45}
Jesse's AirPods #2 21:22:31.386 DEBUG livekit.agents tools execution completed {"speech_id": "speech_1bb9e9c33df6"}
Jesse's AirPods #2 21:22:32.179 INFO livekit.agents LLM metrics
{"model_name": "openai/gpt-4.1-mini", "model_provider": "livekit", "prompt_cached_tokens": 0, "completion_tokens": 26, "tokens_per_second": 32.9}
Jesse's AirPods #2 21:22:32.458 INFO __main__ Time to first audio: 2.950s
Jesse's AirPods #2 21:22:34.183 INFO livekit.agents TTS metrics
{"model_name": "cartesia/sonic-3", "model_provider": "livekit", "audio_duration": 7.01}
Jesse's AirPods #2 <Ctrl+T> Text Mode
```

workshop-demo 0 0 0 Cursor Tab Ln 47, Col 1 Spaces: 4 UTF-8 LF () Python 3.14.0 64-bit

>> Currently it's 49.1 degrees Fahrenheit in San Francisco with clear conditions. >> Cool. What about in Tokyo? Tokyo is chilling at 42.6 degrees Fahrenheit right now with clear skies. >> Nice. >> Anything else I can about um What's the weather like on Mars? >> Mars? I wish I had that weather report for Earthly.

■ [49:39 - 49:59]



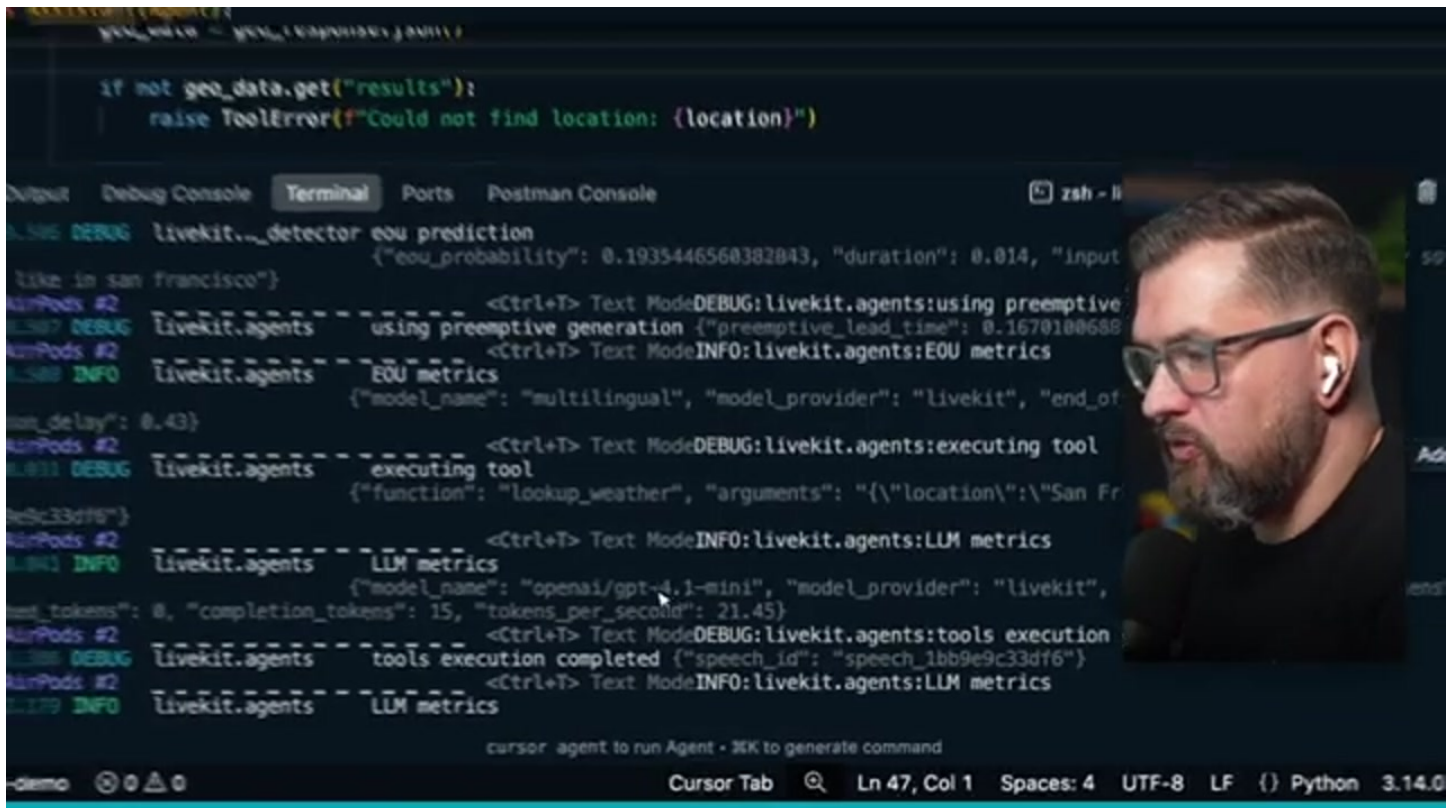
```
agent.py x
livekit-voice-agent > agent.py
19 class Assistant(Agent):
20     you_said = you_response.janus()
47
48     if not geo_data.get("results"):
49         raise ToolError(f"Could not find location: {location}")
50

Problems Output Debug Console Terminal Ports Postman Console
Jesse's AirPods #2 21:22:50.574 DEBUG livekit._detector eou prediction
{"eou_probability": 0.804743230342865, "duration": 0.063, "input": "weather like in san francisco<|im_end|>\n<|im_start|>assistant\ncurrently it's 491°F in san francisco with anything else while you're at it<|im_end|>\n<|im_start|>user\nwhat about in tokyo<|im_end|>\n<|im_start|>assistant\nright now with clear skies anything else i can look up for you<|im_end|>\n<|im_start|>user\nnice about what"}
Jesse's AirPods #2 21:22:50.576 INFO livekit.agents EOU metrics
{"model_name": "multilingual", "model_provider": "livekit", "end_token": 41, "tokens_per_second": 49.76}
"transcription_delay": 0.34}
Jesse's AirPods #2 21:22:51.390 INFO __main__ Time to first audio: 0.815s
Jesse's AirPods #2 21:22:51.401 INFO livekit.agents LLM metrics
{"model_name": "openai/gpt-4.1-mini", "model_provider": "livekit", "end_token": 41, "tokens_per_second": 49.76}
"prompt_cached_tokens": 0, "completion_tokens": 41, "tokens_per_second": 49.76}
Jesse's AirPods #2 21:22:54.163 INFO livekit.agents TTS metrics
{"model_name": "cartesia/sonic-3", "model_provider": "livekit", "end_token": 41, "tokens_per_second": 49.76}
"audio_duration": 11.15}
Jesse's AirPods #2 ----- <Ctrl+T> Text Mode

cursor: agent to run Agent - RK to generate command
workshop-demo 0 0 0 Cursor Tab Ln 47, Col 1 Spaces: 4 UTF-8 LF () Python 3.14.0 64-bit
```

Earthly. I'm your go-to AI, but Martian weather might be. >> All right, it looks like it works. So, if we go down here, we can see that the uh the tool uh did get executed. So, look up weather. it passed the the location there with San Francisco and then it provided us the the the response there. Another really cool tool that we

■ [49:59 - 50:19]



The screenshot shows a Cursor IDE terminal window with a dark theme. The terminal displays logs for a LiveKit agent. At the top, a Python code snippet is visible: `if not geo_data.get("results"): raise ToolError(f"Could not find location: {location}")`. Below this, the terminal tabs include 'Output', 'Debug Console', 'Terminal' (selected), 'Ports', and 'Postman Console'. The logs show the agent detecting EOU (End of Utterance) and making a prediction. It then uses a preemptive generation process with a lead time of 0.1670100688. The agent reports EOU metrics with a model name of 'multilingual' and provider 'livekit'. It then executes a tool named 'lookup_weather' with arguments for 'San Francisco'. The tool execution is completed, and the agent reports LLM metrics for the 'openai/gpt-4.1-mini' model. A status bar at the bottom indicates 'Ln 47, Col 1' and 'Python 3.14.0'. On the right side of the terminal, there is a small video inset of a man with glasses and a beard, wearing a black shirt, speaking into a microphone.

```
if not geo_data.get("results"):
    raise ToolError(f"Could not find location: {location}")

Output Debug Console Terminal Ports Postman Console zsh - li
A.586 DEBUG livekit._detector eou prediction
{"eou_probability": 0.1935446560382043, "duration": 0.014, "input
like in san francisco"}
A.587 DEBUG LiveKit.agents using preemptive generation {"preemptive_lead_time": 0.1670100688
A.588 DEBUG LiveKit.agents using preemptive generation {"preemptive_lead_time": 0.1670100688
A.589 INFO LiveKit.agents EOU metrics
{"model_name": "multilingual", "model_provider": "livekit", "end_of
eou_delay": 0.43}
A.590 DEBUG LiveKit.agents executing tool
{"function": "lookup_weather", "arguments": "{\\"location\\":\\"San Fr
9e9c33df6"}
A.591 INFO LiveKit.agents LLM metrics
{"model_name": "openai/gpt-4.1-mini", "model_provider": "livekit",
next_tokens": 0, "completion_tokens": 15, "tokens_per_second": 21.45}
A.592 DEBUG LiveKit.agents tools execution completed {"speech_id": "speech_1bb9e9c33df6"}
A.593 INFO LiveKit.agents LLM metrics

cursor agent to run Agent - 35K to generate command
demo Cursor Tab Ln 47, Col 1 Spaces: 4 UTF-8 LF {} Python 3.14.0
```

there. Another really cool tool that we can use is MCP or model context protocol. This lets you connect to external tool servers. Now this is useful when you want to share tools across multiple agents. Tools are managed maybe they're managed by separate teams or services or maybe you want to integrate with thirdparty uh MCP

Model Context Protocol (MCP)



LIVEKIT.IO

want to integrate with thirdparty uh MCP compatible systems. So let's connect to the LiveKit documentation MCP server. And so this turns our agent into a voice interface for searching LiveKit docs. So in our terminal, we'll uh UV add LiveKit agents MCP. And let's add another import here from

■ [50:39 - 50:59]

```
voice-agent > agent.py

import logging

from dotenv import load_dotenv
from livekit import agents
from livekit.agents import Agent, AgentServer, AgentSession, JobContext, room_io
from livekit.plugins import noise_cancellation, silero
from livekit.plugins.turn_detector.multilingual import MultilingualModel
from livekit.agents import llm, stt, tts, inference
from livekit.agents import AgentStateChangedEvent, MetricsCollectedEvent, metrics
from livekit.agents import function_tool, RunContext, ToolError
import time
import httpx

logger = logging.getLogger(__name__)

load_dotenv()

class Assistant(Agent):
    def __init__(self) -> None:
        super().__init__(
            instructions=(
                "You are an upbeat, slightly sarcastic voice AI for tech support. "
                "Help the caller fix issues without rambling, and keep replies under 3 sentences. "
```



And let's add another import here from LifeKit agents and that is MCP. And then we'll connect our MCP server to our session using MCP servers. that is going to be under agent session. So let's go find that agent session. We'll add another option here under

■ [50:59 - 51:19]

```
75 async def entrypoint(ctx: SubContext):
76     session = AgentSession(
77         tts=tts.FallbackAdapter(
78             [
79                 inference.TTS.from_model_string(
80                     "cartesia/sonic-3:9626c31c-bec5-4cca-baa8-f8ba9e84c8bc"
81                 ),
82                 inference.TTS.from_model_string("inworld/inworld-tts-1"),
83             ]
84         ),
85         vad=silero.VAD.load(),
86         turn_detection=MultilingualModel(),
87         preemptive_generation=True,
88         mcp_servers=[
89             mcp.MCPServerHTTP(url="https://docs.livekit.io/mcp"),
90         ],
91     )
92
93     usage_collector = metrics.UsageCollector()
94     last_eou_metrics: metrics.EOUMetrics | None = None
95
96     @session.on("metrics_collected")
97     def _on_metrics_collected(ev: MetricsCollectedEvent):
98         nonlocal last_eou_metrics
99         if ev.metrics.type == "eou_metrics":
```



We'll add another option here under agent session MCP servers and then we'll set the MCP server http to docs.livekit.io/mcp. And then lastly, let's go back up to our instructions and let's update these instructions to include everything. So, not only are we a sarcastic um tech

■ [51:19 - 51:39]

```
voice-agent > agent.py
import httpx

logger = logging.getLogger(__name__)

load_dotenv()

class Assistant(Agent):
    def __init__(self) -> None:
        super().__init__(
            instructions=[
                "You are an upbeat, slightly sarcastic voice AI for tech support. "
                "Help the caller fix issues without rambling, and keep replies under 3 sen
                "You can look up the weather if asked. You can also answer questions about
                "LiveKit by searching the documentation. When users ask about LiveKit "
                "features, APIs, or how to build something, use the docs search tools "
                "to find accurate information."
            ],
        )

    @function_tool()
    async def lookup_weather(
        self,
        context: RunContext,
```



not only are we a a sarcastic um tech support person and we we try not to ramble, but we can also look up the weather and we can also answer questions about LifeKit by searching the documentation. When a user asks about LifeKit features, APIs, or how to build something, use the docs search tools to find accurate information. Amazing. So,

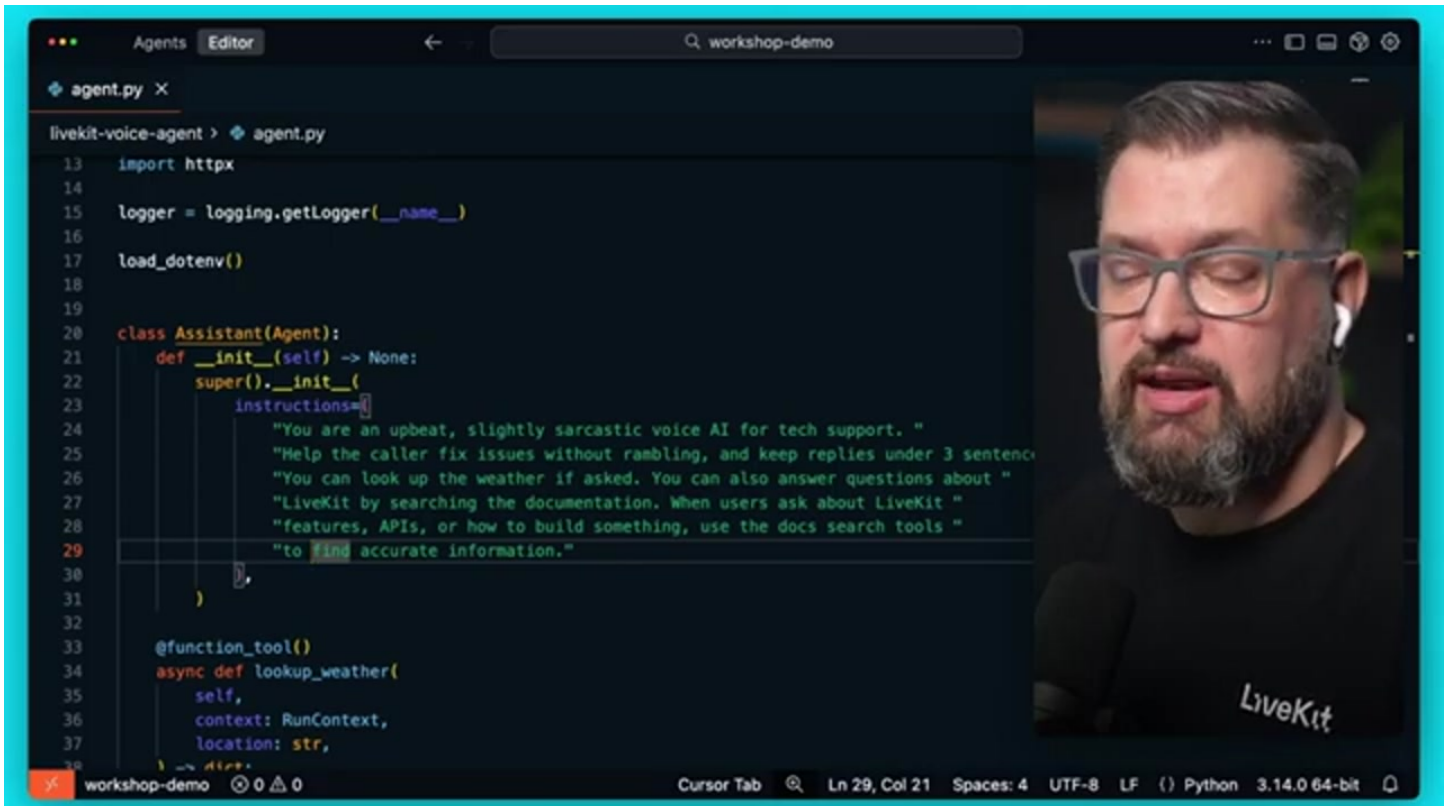
■ [51:39 - 51:59]

```
livekit-voice-agent > agent.py
13 import httpx
14
15 logger = logging.getLogger(__name__)
16
17 load_dotenv()
18
19
20 class Assistant(Agent):
21     def __init__(self) -> None:
22         super().__init__(
23             instructions="""
24             "You are an upbeat, slightly sarcastic voice AI for tech support
25             "Help the caller fix issues without rambling, and keep replies u
26             "You can look up the weather if asked. You can also answer quest
27             "LiveKit by searching the documentation. When users ask about Li
28             "features, APIs, or how to build something, use the docs search
29             "to find accurate information."
30         """,
31         )
32
33     @function_tool()
34     async def lookup_weather(
35         self,
36         context: RunContext,
```



find accurate information. Amazing. So, when the session starts, it connects to the MCP server and fetches a list of the available tools. Each MCP tool then has a name and a description just like the the local function tool methods. So these MCP tools are automatically registered alongside your local tools and the LLM sees all of them together and it knows when to use them I guess

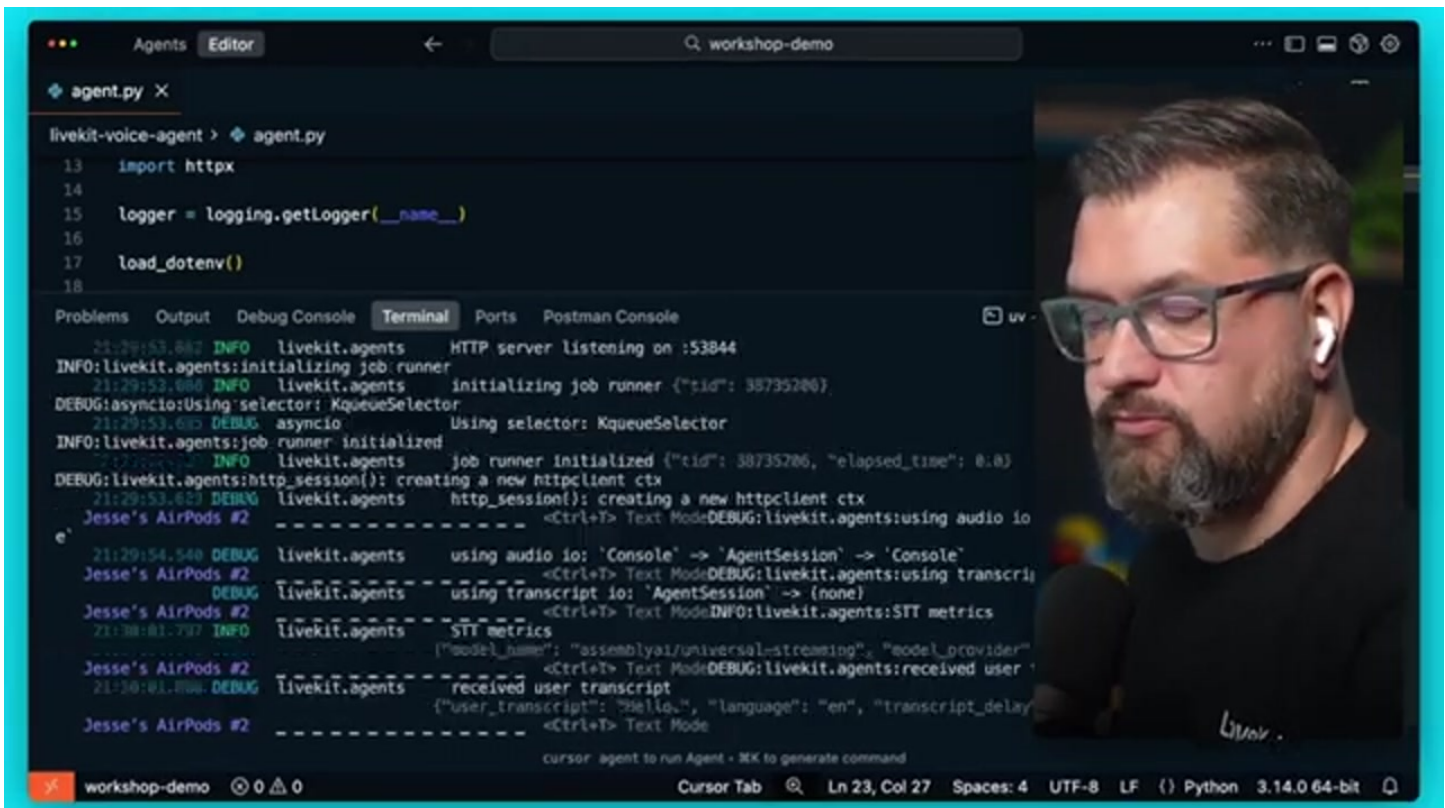
■ [51:59 - 52:19]



```
agent.py X
livekit-voice-agent > agent.py
13 import httpx
14
15 logger = logging.getLogger(__name__)
16
17 load_dotenv()
18
19
20 class Assistant(Agent):
21     def __init__(self) -> None:
22         super().__init__(
23             instructions=[
24                 "You are an upbeat, slightly sarcastic voice AI for tech support. "
25                 "Help the caller fix issues without rambling, and keep replies under 3 sentences. "
26                 "You can look up the weather if asked. You can also answer questions about "
27                 "LiveKit by searching the documentation. When users ask about LiveKit "
28                 "features, APIs, or how to build something, use the docs search tools "
29                 "to find accurate information."
30             ],
31         )
32
33     @function_tool()
34     async def lookup_weather(
35         self,
36         context: RunContext,
37         location: str,
38     ) -> dict:
```

and it knows when to use them I guess somehow. I don't know. No, no, no. It uses the the tool descriptions from the MCP server uh to guide the LLM on when to use each tool. So let's go ahead and test this out. Again, make sure that you have saved this and then let's go ahead and UV run uh agents in the console again. Hello.

[52:19 - 52:39]



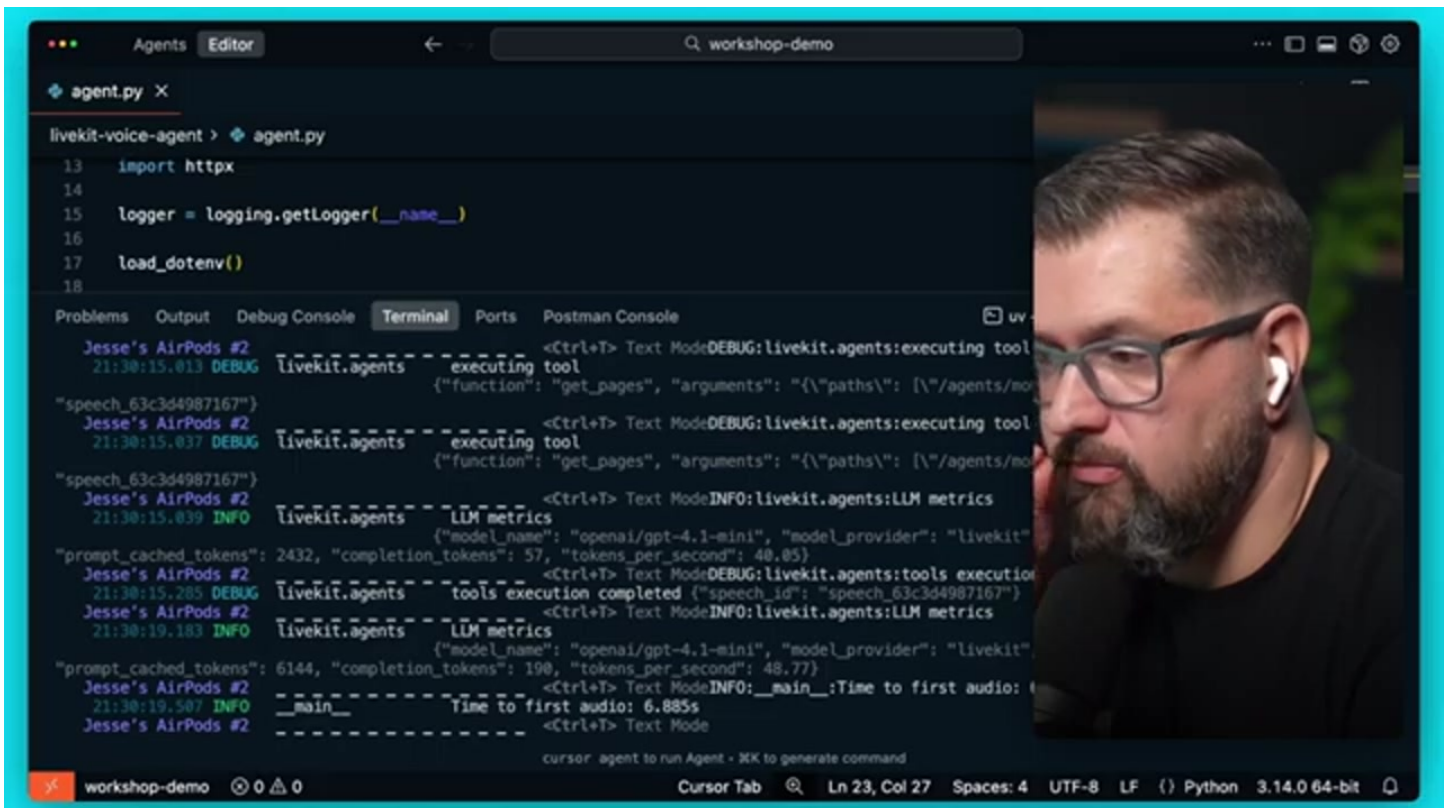
```
livekit-voice-agent > agent.py
13 import httpx
14
15 logger = logging.getLogger(__name__)
16
17 load_dotenv()
18

21:29:53.882 INFO livekit.agents HTTP server listening on :53844
INFO:livekit.agents:initializing job runner
21:29:53.888 INFO livekit.agents initializing job runner ("id": 38735286)
DEBUG:asyncio:Using selector: KqueueSelector
21:29:53.893 DEBUG asyncio Using selector: KqueueSelector
INFO:livekit.agents:job runner initialized
21:29:53.927 INFO livekit.agents job runner initialized ("cid": 38735286, "elapsed_time": 0.0)
DEBUG:livekit.agents:http_session(): creating a new httpclient ctx
21:29:53.929 DEBUG livekit.agents http_session(): creating a new httpclient ctx
Jesse's AirPods #2 ----- <Ctrl+T> Text ModeDEBUG:livekit.agents:using audio io
21:29:54.540 DEBUG livekit.agents using audio io: 'Console' -> 'AgentSession' -> 'Console'
Jesse's AirPods #2 ----- <Ctrl+T> Text ModeDEBUG:livekit.agents:using transcrip
DEBUG livekit.agents using transcript io: 'AgentSession' -> (none)
Jesse's AirPods #2 ----- <Ctrl+T> Text ModeINFO:livekit.agents:STT metrics
21:30:01.797 INFO livekit.agents STT metrics
["model_name": "assemblyai/universal-streaming", "model_provider"
Jesse's AirPods #2 ----- <Ctrl+T> Text ModeDEBUG:livekit.agents:received user
21:30:01.898 DEBUG livekit.agents received user transcript
{"user_transcript": "Hello.", "language": "en", "transcript_delay
Jesse's AirPods #2 ----- <Ctrl+T> Text Mode

cursor: agent to run Agent - RK to generate command
```

Hello. >> Hey there. How can I help you with tech or live kit today? >> Yeah. So, um what uh what ST providers does live kit support?

■ [52:39 - 52:59]



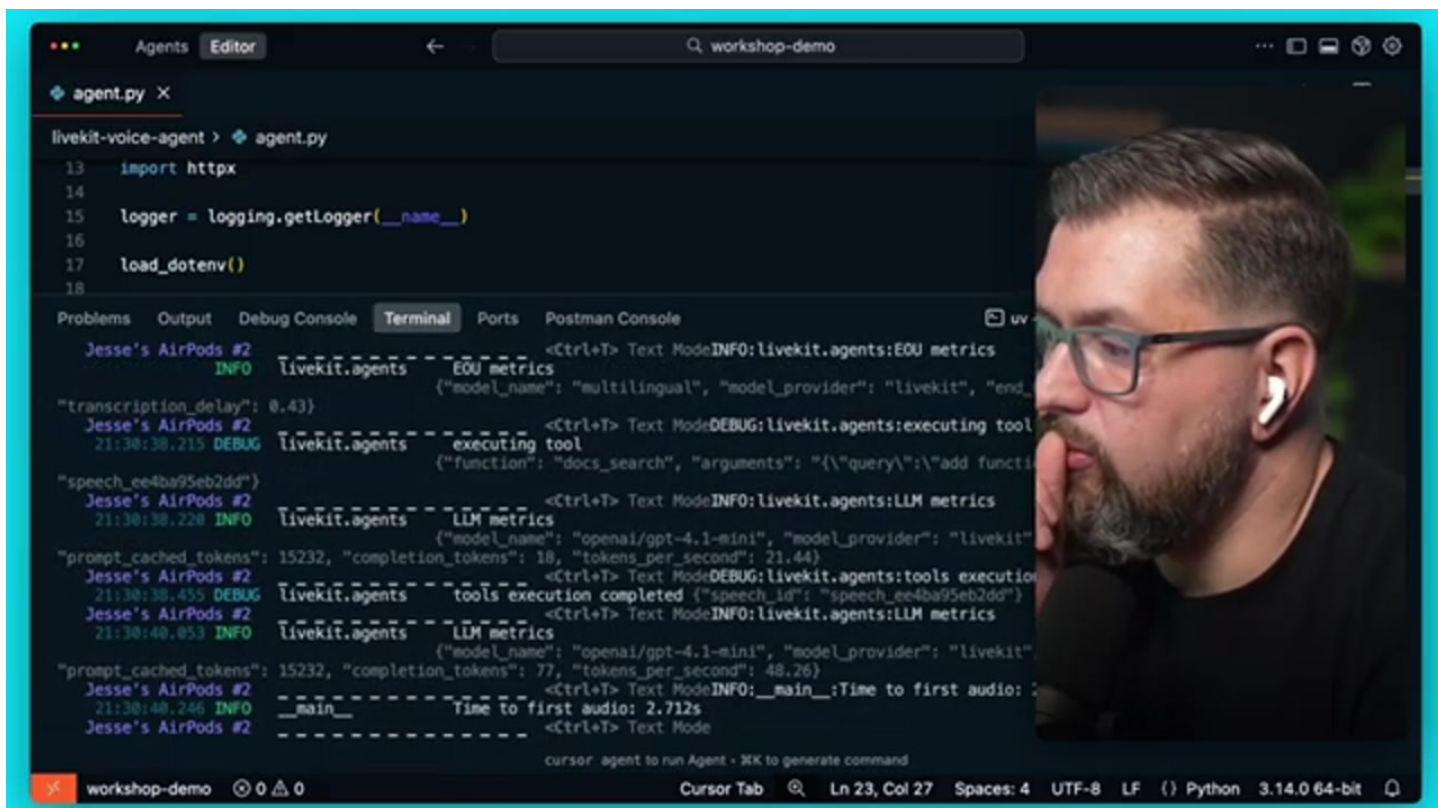
```
agent.py X
livekit-voice-agent > agent.py
13 import httpx
14
15 logger = logging.getLogger(__name__)
16
17 load_dotenv()
18

Problems Output Debug Console Terminal Ports Postman Console
Jesse's AirPods #2 21:30:15.013 DEBUG livekit.agents executing tool
{"function": "get_pages", "arguments": "{\\"paths\\": [\\"/agents/mo
"speech_63c3d4987167"}
Jesse's AirPods #2 21:30:15.037 DEBUG livekit.agents executing tool
{"function": "get_pages", "arguments": "{\\"paths\\": [\\"/agents/mo
"speech_63c3d4987167"}
Jesse's AirPods #2 21:30:15.039 INFO livekit.agents LLM metrics
{"model_name": "openai/gpt-4.1-mini", "model_provider": "livekit"
"prompt_cached_tokens": 2432, "completion_tokens": 57, "tokens_per_second": 48.05}
Jesse's AirPods #2 21:30:15.205 DEBUG livekit.agents tools execution completed {"speech_id": "speech_63c3d4987167"}
Jesse's AirPods #2 21:30:19.183 INFO livekit.agents LLM metrics
{"model_name": "openai/gpt-4.1-mini", "model_provider": "livekit"
"prompt_cached_tokens": 6144, "completion_tokens": 190, "tokens_per_second": 48.77}
Jesse's AirPods #2 21:30:19.507 INFO __main__ Time to first audio: 6.885s
Jesse's AirPods #2 21:30:19.507 INFO __main__ Time to first audio: 6.885s

cursor: agent to run Agent - KK to generate command
workshop-demo 0 0 0 Cursor Tab Ln 23, Col 27 Spaces: 4 UTF-8 LF () Python 3.14.0 64-bit
```

LiveKit supports these speech-to-text SDP providers via LiveKit inference cloud service assembly AI English and six multilingual languages Cartisia Inc. Whisper 98 languages deepgram gram several English and multilingual models 11. >> How do I add a function tool to my agent?

■ [52:59 - 53:19]



```
agent.py X
livekit-voice-agent > agent.py
13 import httpx
14
15 logger = logging.getLogger(__name__)
16
17 load_dotenv()
18

Problems Output Debug Console Terminal Ports Postman Console
Jesse's AirPods #2 <Ctrl+T> Text ModeINFO:livekit.agents:EOU metrics
INFO livekit.agents EOU metrics
{"model_name": "multilingual", "model_provider": "livekit", "end_
"transcription_delay": 0.43}
Jesse's AirPods #2 <Ctrl+T> Text ModeDEBUG:livekit.agents:executing tool
21:30:38.215 DEBUG livekit.agents executing tool
{"function": "docs_search", "arguments": "{\\"query\\":\\"add functi
"speech_id": "ee4ba95eb2dd"}
Jesse's AirPods #2 <Ctrl+T> Text ModeINFO:livekit.agents:LLM metrics
21:30:38.228 INFO livekit.agents LLM metrics
{"model_name": "openai/gpt-4.1-mini", "model_provider": "livekit"
"prompt_cached_tokens": 15232, "completion_tokens": 18, "tokens_per_second": 21.44}
Jesse's AirPods #2 <Ctrl+T> Text ModeDEBUG:livekit.agents:tools execution
21:30:38.455 DEBUG livekit.agents tools execution completed {"speech_id": "speech_id": "ee4ba95eb2dd"}
Jesse's AirPods #2 <Ctrl+T> Text ModeINFO:livekit.agents:LLM metrics
21:30:40.053 INFO livekit.agents LLM metrics
{"model_name": "openai/gpt-4.1-mini", "model_provider": "livekit"
"prompt_cached_tokens": 15232, "completion_tokens": 77, "tokens_per_second": 48.26}
Jesse's AirPods #2 <Ctrl+T> Text ModeINFO:livekit.agents:Time to first audio:
21:30:48.246 INFO __main__ Time to first audio: 2.712s
Jesse's AirPods #2 <Ctrl+T> Text Mode
cursor: agent to run Agent - XK to generate command
workshop-demo 0 0 0 Cursor Tab Ln 23, Col 27 Spaces: 4 UTF-8 LF () Python 3.14.0 64-bit
```

To add a function tool to your agent in LiveKit, you use the `@function_tool` decorator on a method in your agent class. This lets the LLM access. >> We can see here in the terminal executing tool, the tool name is docs search and uh speech to text providers is what it passed in and so it used the tool and then uh here's tool execution completed. Uh it executed another tool

■ [53:19 - 53:39]

```
13 import httpx
14
15 logger = logging.getLogger(__name__)
16
17 load_dotenv()
18
```

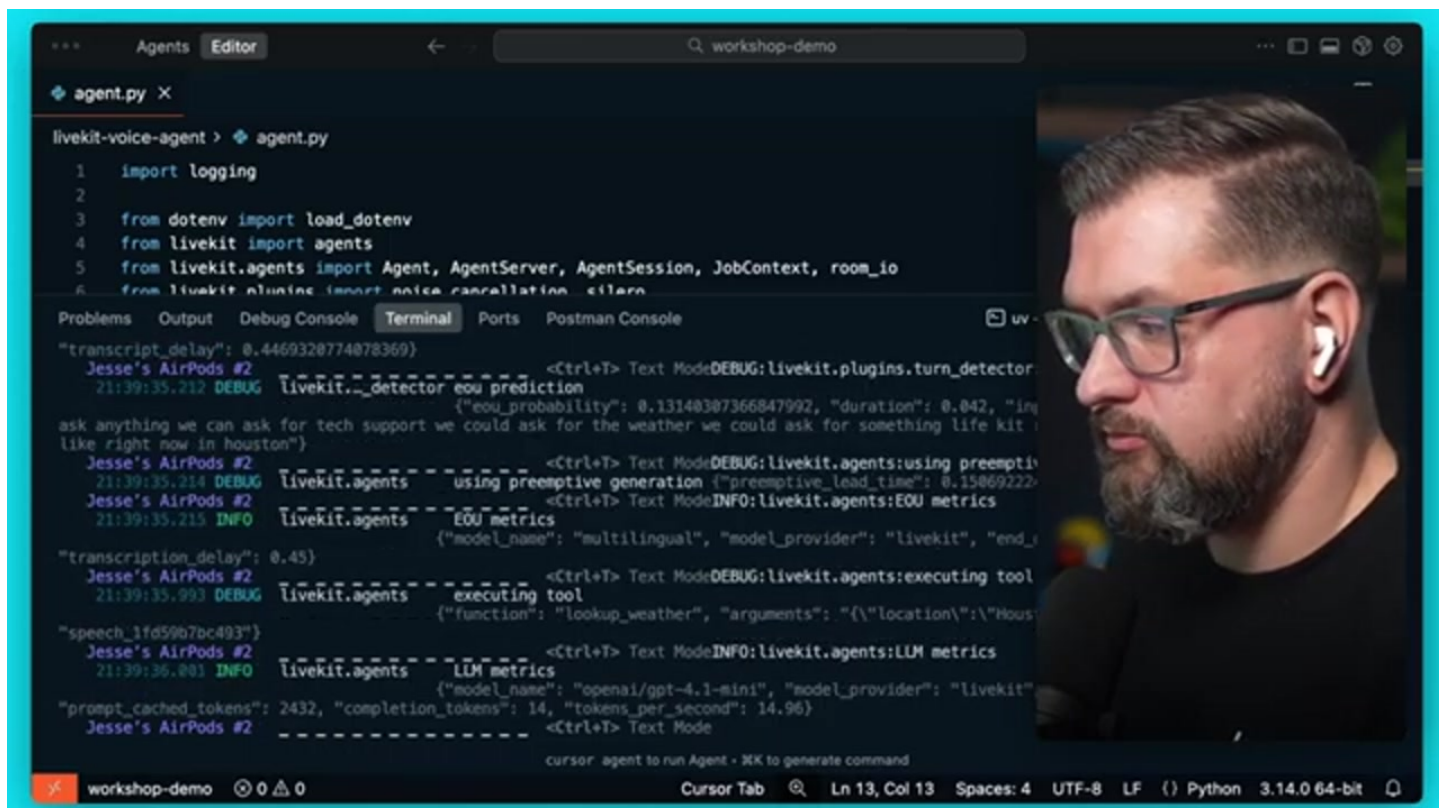
Problems Output Debug Console Terminal Ports Postman Console

INFO livekit.agents EOU metrics
{"model_name": "multilingual", "model_provider": "livekit"}
"transcription_delay": 0.63}
Jesse's AirPods #2 <Ctrl+T> Text ModeDEBUG:livekit.agents:executing tool
21:30:13.310 DEBUG livekit.agents {"function": "docs_search", "arguments": "{\\"query\\":\\""}
"speech_63c3d4987167"}<Ctrl+T> Text ModeINFO:livekit.agents:LLM metrics
Jesse's AirPods #2 {"model_name": "openai/gpt-4.1-mini", "model_provider": "livekit"}
21:30:13.315 INFO livekit.agents {"tokens_per_second": 23.41}
"prompt_cached_tokens": 0, "completion_tokens": 17,
Jesse's AirPods #2 <Ctrl+T> Text ModeDEBUG:livekit.agents:tools execution completed {"speech_id": "speech_63c3d4987167"}
21:30:13.613 DEBUG livekit.agents {"function": "get_pages", "arguments": "{\\"paths\\": [\\""]}
Jesse's AirPods #2 <Ctrl+T> Text ModeDEBUG:livekit.agents:executing tool
21:30:15.013 DEBUG livekit.agents {"function": "get_pages", "arguments": "{\\"paths\\": [\\"/agents/models/stt/#plug..."]}
"speech_63c3d4987167"}<Ctrl+T> Text ModeDEBUG:livekit.agents:executing tool
Jesse's AirPods #2 {"function": "get_pages", "arguments": "{\\"paths\\": [\\"/agents/models/stt/#plug..."]}
21:30:15.037 DEBUG livekit.agents {"function": "get_pages", "arguments": "{\\"paths\\": [\\"/agents/models/stt/#plug..."]}
"speech_63c3d4987167"}<Ctrl+T> Text ModeDEBUG:livekit.agents:executing tool
Jesse's AirPods #2 {"function": "get_pages", "arguments": "{\\"paths\\": [\\"/agents/models/stt/#plug..."]}
21:30:15.037 DEBUG livekit.agents {"function": "get_pages", "arguments": "{\\"paths\\": [\\"/agents/models/stt/#plug..."]}

cursor agent to run Agent • %K to generate command

completed. Uh it executed another tool get pages and so we can see it is using our MCP server perfectly. So now we could ask um anything. We could ask for tech support. We could ask for the weather. We could ask for something LifeKit related. So let's say um what is the weather like right now in Houston?

■ [53:39 - 53:59]

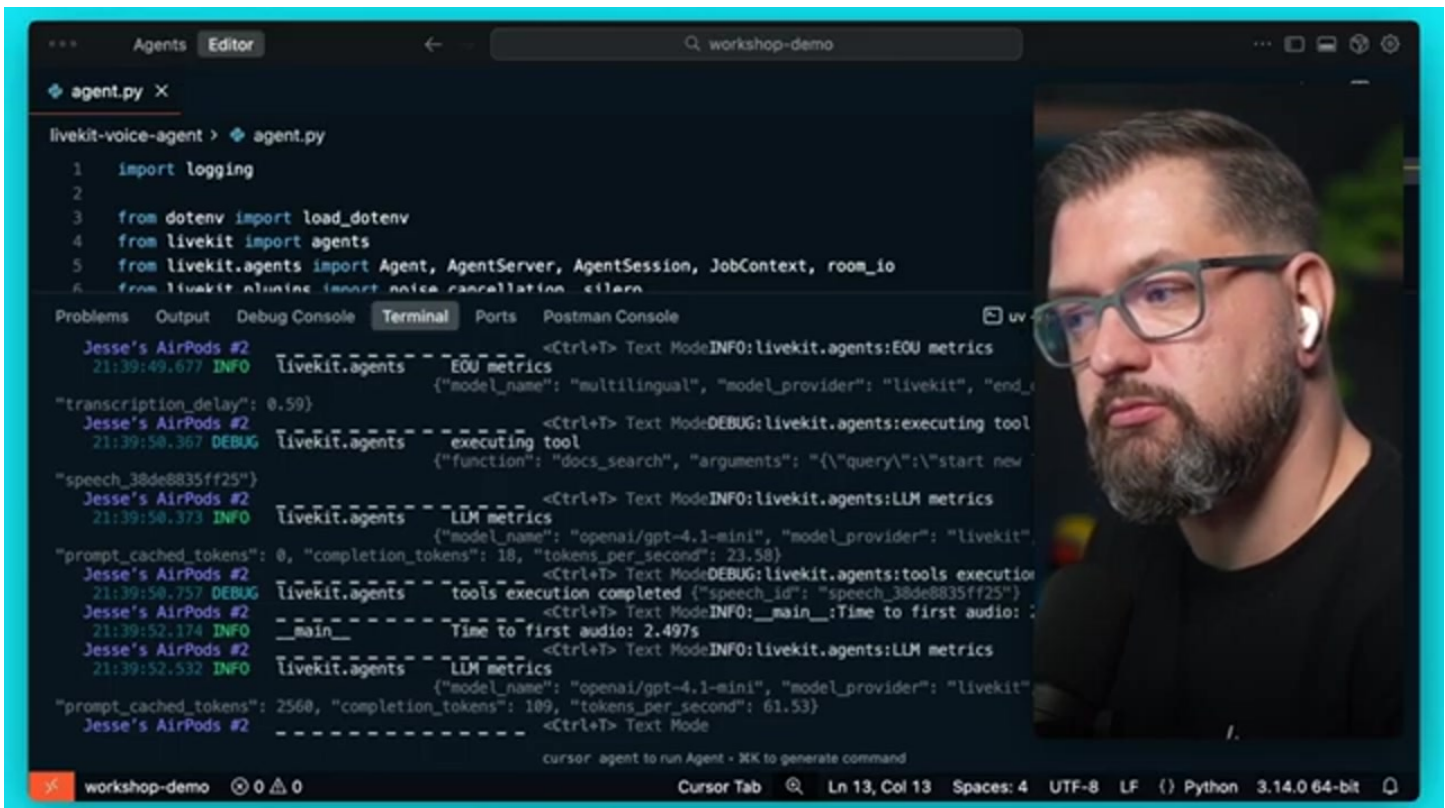


```
... Agents Editor workshop-demo
agent.py x
livekit-voice-agent > agent.py
1 import logging
2
3 from dotenv import load_dotenv
4 from livekit import agents
5 from livekit.agents import Agent, AgentServer, AgentSession, JobContext, room_io
6 from livekit.plugins import noise_cancellation, silence

Problems Output Debug Console Terminal Ports Postman Console uv
"transcription_delay": 0.4469328774878369}
Jesse's AirPods #2 <Ctrl+T> Text ModeDEBUG:livekit.plugins.turn_detector
21:39:35.212 DEBUG livekit._detector eou prediction
{"eou_probability": 0.13140387366847992, "duration": 0.042, "in
ask anything we can ask for tech support we could ask for the weather we could ask for something life kit
like right now in houston")
Jesse's AirPods #2 <Ctrl+T> Text ModeDEBUG:livekit.agents:using preempti
21:39:35.214 DEBUG livekit.agents using preemptive generation ("preemptive_lead_time": 0.15869222)
Jesse's AirPods #2 <Ctrl+T> Text ModeINFO:livekit.agents:Eou metrics
21:39:35.215 INFO livekit.agents EOU metrics
{"model_name": "multilingual", "model_provider": "livekit", "end_
"transcription_delay": 0.45}
Jesse's AirPods #2 <Ctrl+T> Text ModeDEBUG:livekit.agents:executing tool
21:39:35.993 DEBUG livekit.agents executing tool
{"function": "lookup_weather", "arguments": "{\\"location\\":\\"Hous
"speech_id59b7bc483"}
Jesse's AirPods #2 <Ctrl+T> Text ModeINFO:livekit.agents:LLM metrics
21:39:36.001 INFO livekit.agents LLM metrics
{"model_name": "openai/gpt-4.1-mini", "model_provider": "livekit"
"prompt_cached_tokens": 2432, "completion_tokens": 14, "tokens_per_second": 14.96}
Jesse's AirPods #2 <Ctrl+T> Text Mode
cursor: agent to run Agent - XK to generate command
workshop-demo 0 0 0 Cursor Tab Ln 13, Col 13 Spaces: 4 UTF-8 LF () Python 3.14.0 64-bit
```

Houston? >> The weather in Houston right now is around 70.4 degrees F and conditions are clear. Anything else you'd like to know? >> Yeah. Yeah. How can I quickly start up a new LiveKit agent? To quickly start a new LiveKit agent, your agent code first registers as an

■ [53:59 - 54:19]



```
*** Agents Editor *** workshop-demo

agent.py X

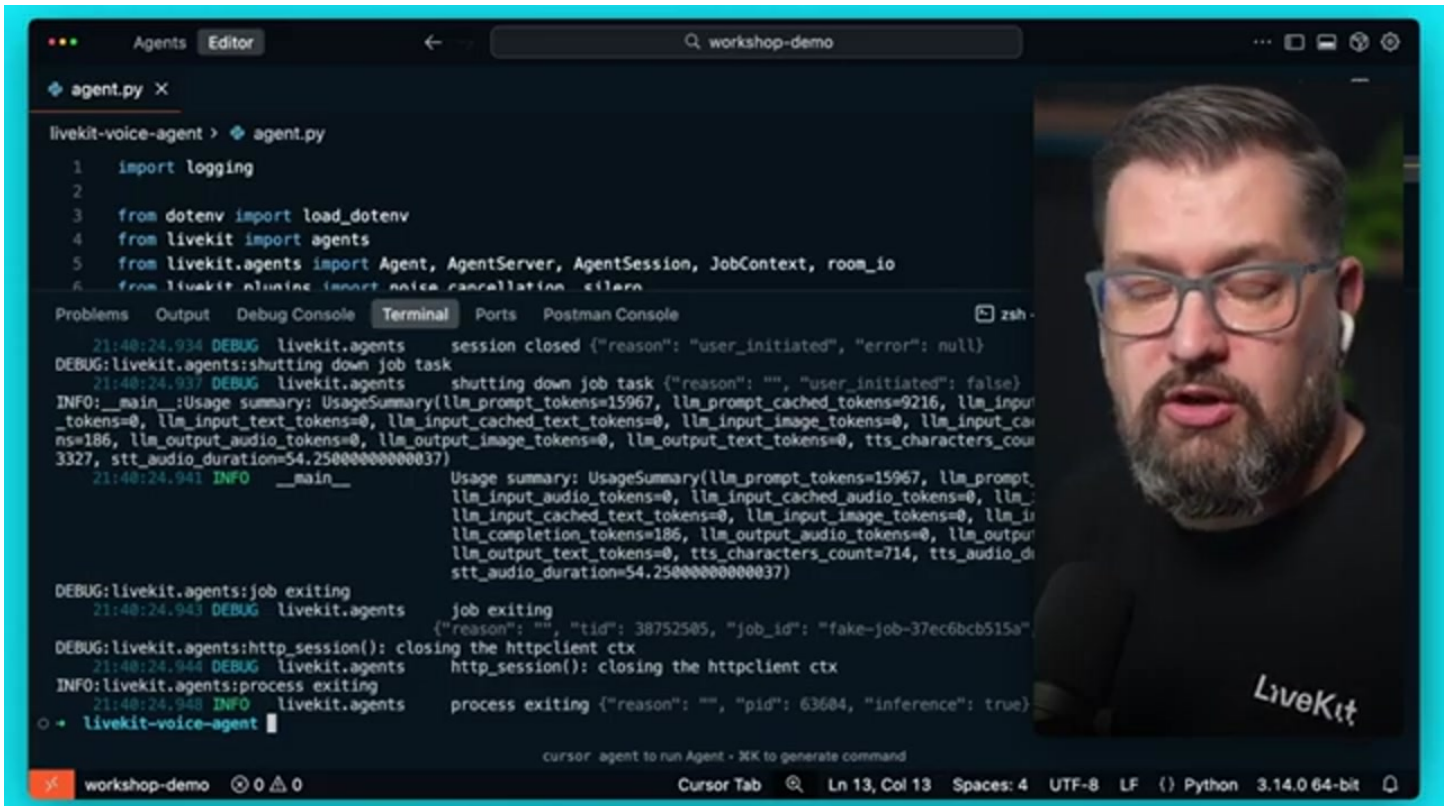
livekit-voice-agent > agent.py
1 import logging
2
3 from dotenv import load_dotenv
4 from livekit import agents
5 from livekit.agents import Agent, AgentServer, AgentSession, JobContext, room_io
6 from livekit.plugins import noise_cancellation, silence

Problems Output Debug Console Terminal Ports Postman Console
Jesse's AirPods #2 21:39:49.677 INFO livekit.agents EOU metrics
{"model_name": "multilingual", "model_provider": "livekit", "end_
"transcription_delay": 0.59}
Jesse's AirPods #2 21:39:50.367 DEBUG livekit.agents executing tool
{"function": "docs_search", "arguments": "{\\"query\\":\\"start new
"speech_38de8835ff25")}
Jesse's AirPods #2 21:39:50.373 INFO livekit.agents LLM metrics
{"model_name": "openai/gpt-4.1-mini", "model_provider": "livekit",
"prompt_cached_tokens": 0, "completion_tokens": 18, "tokens_per_second": 23.58}
Jesse's AirPods #2 21:39:50.757 DEBUG livekit.agents tools execution
livekit.agents tools execution completed {"speech_id": "speech_38de8835ff25"}
Jesse's AirPods #2 21:39:52.174 INFO __main__ Time to first audio: 2.497s
Jesse's AirPods #2 21:39:52.532 INFO livekit.agents LLM metrics
{"model_name": "openai/gpt-4.1-mini", "model_provider": "livekit",
"prompt_cached_tokens": 2560, "completion_tokens": 109, "tokens_per_second": 61.53}
Jesse's AirPods #2 <Ctrl+T> Text Mode

cursor: agent to run Agent - RK to generate command
workshop-demo 0 0 0 Cursor Tab Ln 13, Col 13 Spaces: 4 UTF-8 LF () Python 3.14.0 64-bit
```

To quickly start a new LiveKit agent, your agent code first registers as an agent server process with the LiveKit server and waits for requests. When a user connects to a room, LiveKit dispatches a job request to an available agent server, which starts a new agent session process to handle the job. >> Regular explanation. Thank you so much for that. We'll talk to you later. Cool. So now the agent selects the right tool for each question. And the console logs,

■ [54:19 - 54:39]



```
agent.py x
livekit-voice-agent > agent.py
1 import logging
2
3 from dotenv import load_dotenv
4 from livekit import agents
5 from livekit.agents import Agent, AgentServer, AgentSession, JobContext, room_io
6 from livekit.plugins import noise_cancellation, silence_suppression

21:40:24.934 DEBUG livekit.agents session closed {"reason": "user_initiated", "error": null}
DEBUG:livekit.agents:shutting down job task
21:40:24.937 DEBUG livekit.agents shutting down job task {"reason": "", "user_initiated": false}
INFO: __main__: Usage summary: UsageSummary(llm_prompt_tokens=15967, llm_prompt_cached_tokens=9216, llm_input_tokens=0, llm_input_text_tokens=0, llm_input_cached_text_tokens=0, llm_input_image_tokens=0, llm_input_cached_image_tokens=0, llm_output_audio_tokens=0, llm_output_image_tokens=0, llm_output_text_tokens=0, tts_characters_count=186, llm_output_audio_duration=54.250000000000037)
21:40:24.941 INFO __main__ Usage summary: UsageSummary(llm_prompt_tokens=15967, llm_prompt_cached_tokens=9216, llm_input_audio_tokens=0, llm_input_cached_audio_tokens=0, llm_input_cached_text_tokens=0, llm_input_image_tokens=0, llm_input_cached_image_tokens=0, llm_completion_tokens=186, llm_output_audio_tokens=0, llm_output_text_tokens=0, tts_characters_count=714, tts_audio_duration=54.250000000000037)
DEBUG:livekit.agents:job exiting
21:40:24.943 DEBUG livekit.agents job exiting {"reason": "", "tid": 38752505, "job_id": "fake-job-37ec6bcb515a"}
DEBUG:livekit.agents:http_session(): closing the httpclient ctx
21:40:24.944 DEBUG livekit.agents http_session(): closing the httpclient ctx
INFO:livekit.agents:process exiting
21:40:24.948 INFO livekit.agents process exiting {"reason": "", "pid": 63604, "inference": true}
o • livekit-voice-agent
```

workshop-demo 0 0 0 Cursor Tab Ln 13, Col 13 Spaces: 4 UTF-8 LF () Python 3.14.0 64-bit

for each question. And the console logs, you can search through those and see which tool is being called and with which parameters. So let's take a look at a few best practices for tools. So be specific about what the tool does and when to use it. Uh keep tools fast under two seconds ideally or provide user feedback during long operations and then

Tool best practices

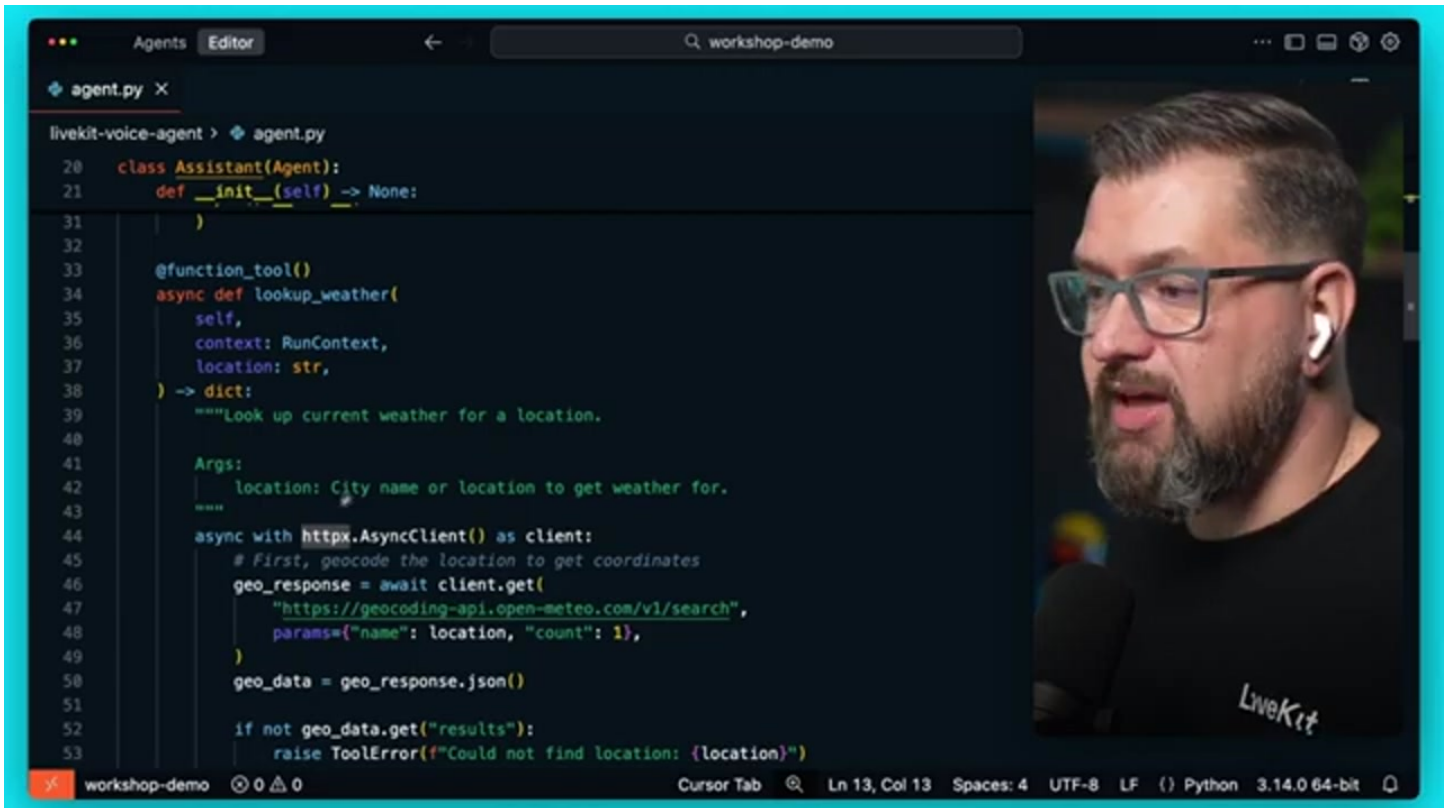
- Be specific in descriptions
- Keep tools fast (<2 seconds)
- Handle errors with ToolError



LIVEKIT.IO

feedback during long operations and then handle errors gracefully with the tool error exception and return meaningful data that helps the LLM form a good response. Now, some tools take a long time and and for those you'll want to give the user feedback and handle potential interruptions. So, to do that, um what we could do in

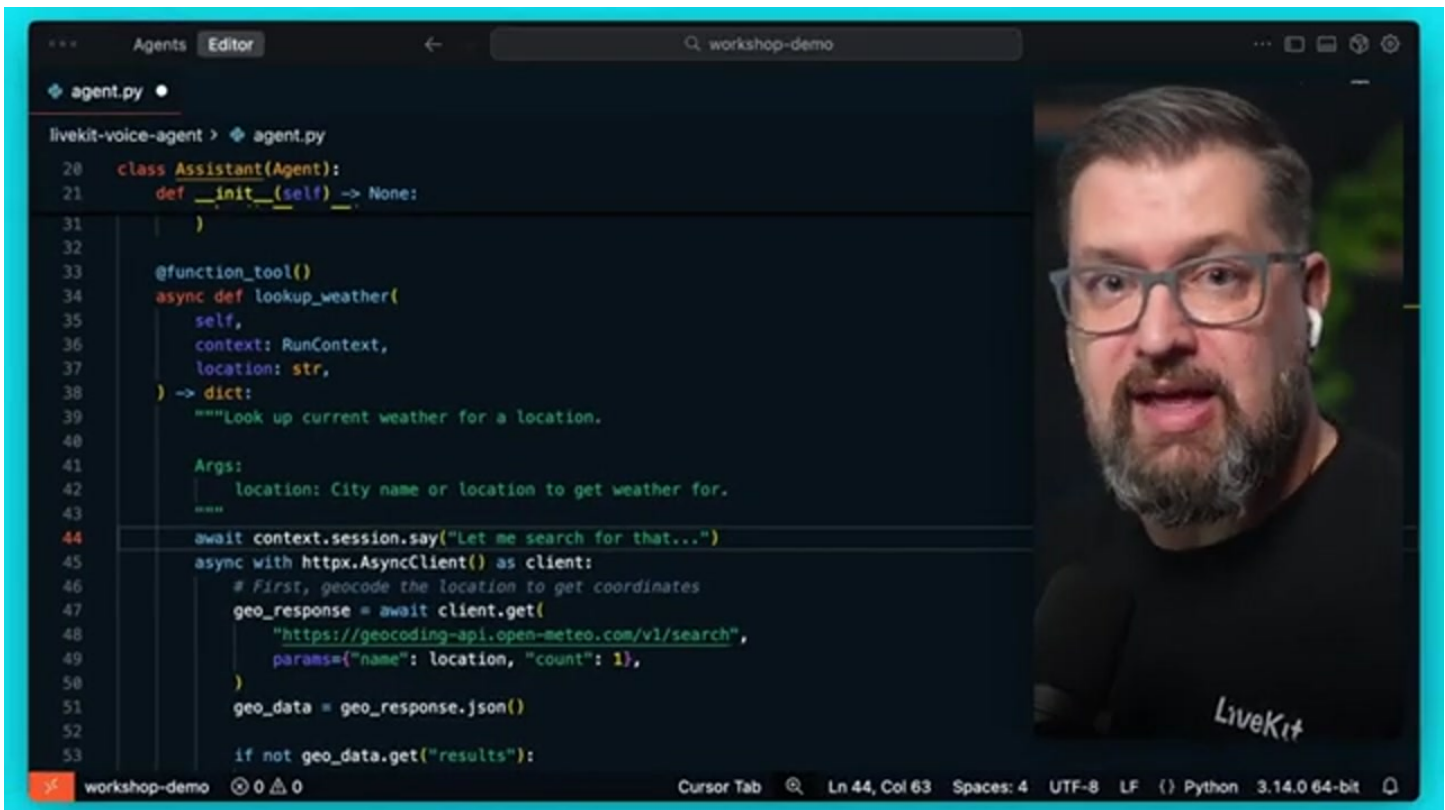
■ [54:59 - 55:19]



```
livekit-voice-agent > agent.py
20 class Assistant(Agent):
21     def __init__(self) -> None:
31
32
33     @function_tool()
34     async def lookup_weather(
35         self,
36         context: RunContext,
37         location: str,
38     ) -> dict:
39         """Look up current weather for a location.
40
41         Args:
42             location: City name or location to get weather for.
43         """
44         async with httpx.AsyncClient() as client:
45             # First, geocode the location to get coordinates
46             geo_response = await client.get(
47                 "https://geocoding-api.open-meteo.com/v1/search",
48                 params={"name": location, "count": 1},
49             )
50             geo_data = geo_response.json()
51
52             if not geo_data.get("results"):
53                 raise ToolError(f"Could not find location: {location}")
```

So, to do that, um what we could do in any function tool, um we could add to that await context session say, let me search for that. So, we're adding that extra context. We're giving the user um confirmation that we're doing something. Now if your tool takes external actions that can't be rolled back uh then we can

■ [55:19 - 55:39]



```
livekit-voice-agent > agent.py
20 class Assistant(Agent):
21     def __init__(self) -> None:
31         )
32
33     @function_tool()
34     async def lookup_weather(
35         self,
36         context: RunContext,
37         location: str,
38     ) -> dict:
39         """Look up current weather for a location.
40
41         Args:
42             location: City name or location to get weather for.
43         """
44         await context.session.say("Let me search for that...")
45         async with httpx.AsyncClient() as client:
46             # First, geocode the location to get coordinates
47             geo_response = await client.get(
48                 "https://geocoding-api.open-meteo.com/v1/search",
49                 params={"name": location, "count": 1},
50             )
51             geo_data = geo_response.json()
52
53             if not geo_data.get("results"):
```

that can't be rolled back uh then we can disable interruptions at the start. For instance, if um your tool creates an order uh is processing a payment, well that can't be undone. We don't want the user to interrupt that part of the agent session. And so for that we could go into any tool again uh just like we did

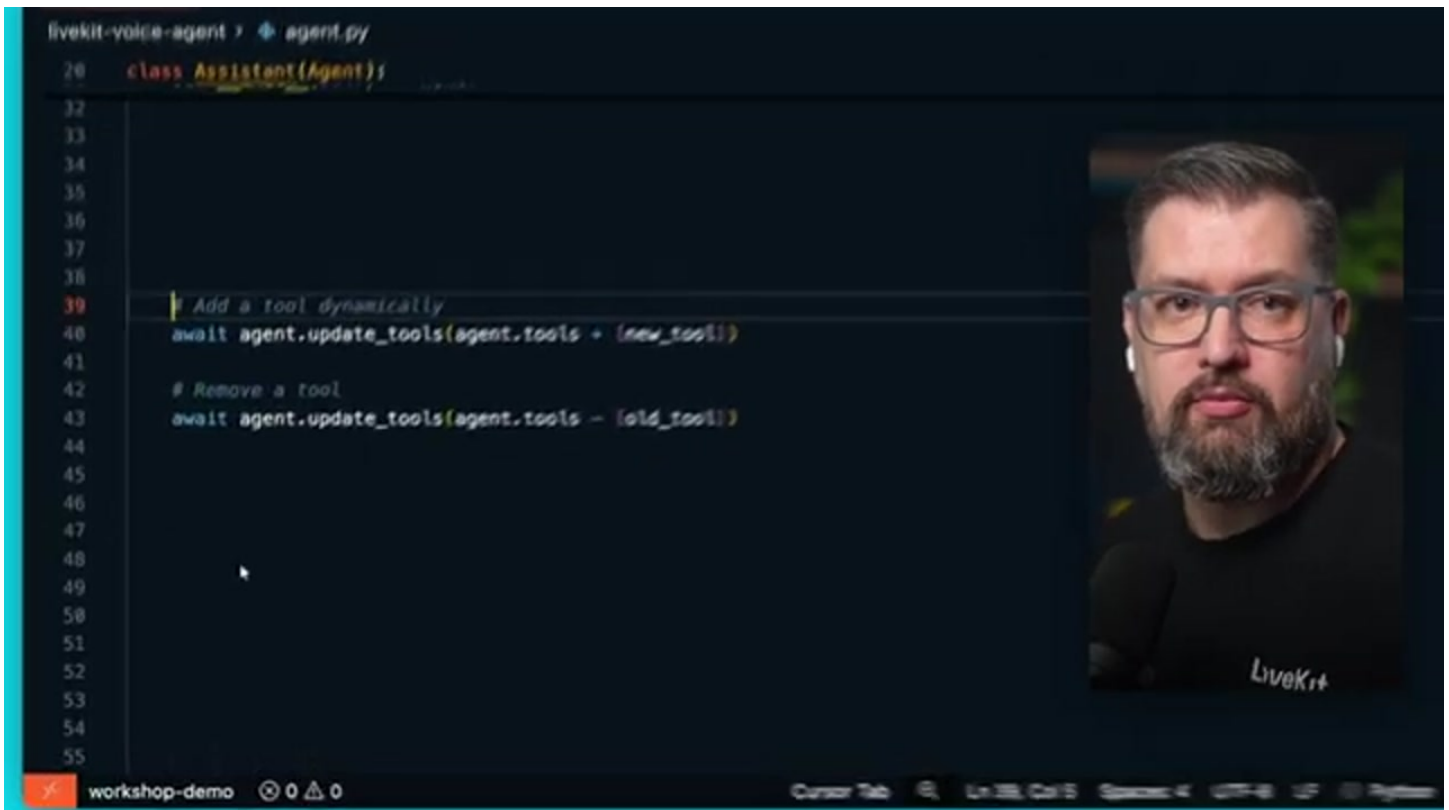
■ [55:39 - 55:59]

```
vekit-voice-agent > agent.py
20 class Assistant(Agent):
21     def __init__(self) -> None:
31         )
32
33     @function_tool()
34     async def lookup_weather(
35         self,
36         context: RunContext,
37         location: str,
38     ) -> dict:
39         """Look up current weather for a location.
40
41         Args:
42             location: City name or location to get weather for.
43         """
44         await context.session.say("Let me search for that...")
45         async with httpx.AsyncClient() as client:
46             # First, geocode the location to get coordinates
47             geo_response = await client.get(
48                 "https://geocoding-api.open-meteo.com/v1/search",
49                 params={"name": location, "count": 1},
50             )
51             geo_data = geo_response.json()
```



into any tool again uh just like we did here with this await context say we can do context.allow disallow interruptions. So that will prevent any interruptions once this tool is called. Uh there could be no interruptions until the tool returns. You can also add or remove tools at runtime based on conversation state.

■ [55:59 - 56:19]



The screenshot shows a video call interface. On the left, a code editor displays the following Python code in a dark theme:

```
livekit-voice-agent > agent.py
20 class Assistant(Agent):
32
33
34
35
36
37
38
39 | Add a tool dynamically
40 await agent.update_tools(agent.tools + [new_tool])
41
42 # Remove a tool
43 await agent.update_tools(agent.tools - [old_tool])
44
45
46
47
48
49
50
51
52
53
54
55
```

On the right, a video feed shows a man with a beard and glasses, wearing a black t-shirt with the 'LiveKit' logo. The video call controls at the bottom include a 'workshop-demo' tab, a mute icon, a video icon, and a 'Cursor Tab' label.

runtime based on conversation state. Here's what that could look like. We could add a tool dynamically. We could remove a tool. So, this is useful for progressive disclosure where you're only showing certain tools after a user has been authenticated or reached a certain point in a workflow. Your agent can now call external APIs with function tools, connect to MCP servers for shared

- ✓ Function tools
- ✓ MCP integration



LIVEKIT.IO

tools, connect to MCP servers for shared capabilities, and handle errors gracefully. And the LLMs automatically select the right tool based on what the user asks. In the final lesson, you're going to build production workflows for collecting consent and escalating to human agents. Your agent has tools. It